

description: "PHẢI dùng khi: vừa sửa agent/skill/template/script dùng chung (bất kỳ file dưới ~/.claude/agents, commands, shared, templates, references, evals, hooks-kztek, scripts) và cần commit+push về repo config KZTEK — kể cả khi đang làm ở project khác. Cũng dùng khi: muốn kéo bản config mới nhất về máy này, kiểm tra config dùng chung có thay đổi chưa commit, hoặc nghi ngờ junction bị đứt sau khi di chuyển repo. KHÔNG dùng khi: chỉ sửa file thuộc riêng project hiện tại (src/, docs/plans/, code-graph/ của project đó — commit bằng git thường)."

/sync-global — Đồng bộ config KZTEK dùng chung

Vấn đề skill này giải quyết

Config dùng chung nằm ở repo Desktop\Claude-Git\claude, được junction vào ~/.claude. Khi bạn đang làm ở **project B** và sửa ~/.claude/agents/cto.md, file thật thuộc **repo config** — git status trong project B **không thấy gì**. Thay đổi dễ nằm chờ rồi bị quên. Skill này commit + push đúng repo, chạy được từ bất kỳ thư mục nào.

Biến

```
REPO = C:/Users/nguye/Desktop/Claude-Git/claude
```

Mọi lệnh git dùng git -C "\$REPO" — KHÔNG cd, để không phá working directory của session hiện tại.

Quy trình

Bước 1 — Kiểm tra junction còn nguyên

```
ls "C:/Users/nguye/.claude/agents/cto.md" >/dev/null 2>&1 || echo "JUNCTION ĐỨT"
```

Nếu đứt → báo user chạy powershell -ExecutionPolicy Bypass -File "\$REPO/scripts/link-global.ps1" rồi DỪNG.

Bước 2 — Xem có gì thay đổi

```
git -C "$REPO" status --short
git -C "$REPO" diff --stat
```

- Không có thay đổi nào → báo "Config đã sạch", nhảy tới Bước 5 (chỉ pull).
- Có thay đổi → hiển thị git -C "\$REPO" diff cho user xem TRƯỚC khi commit.

Bước 3 — Pull trước khi commit (tránh conflict về sau)

```
git -C "$REPO" stash push -m "sync-global tạm" --include-untracked # chỉ khi có thay đổi
git -C "$REPO" pull --rebase
git -C "$REPO" stash pop # chỉ khi đã stash
```

Nếu conflict → DỪNG, báo user, KHÔNG tự resolve.

Bước 4 — Commit

Stage theo **đường dẫn cụ thể**, TUYỆT ĐỐI KHÔNG `git add -A` (repo có thể đang chứa thay đổi dở dang khác của user ở `scripts/`, `docs/`).

```
git -C "$REPO" add <đường-dẫn-cụ-thể-tùng-file>
git -C "$REPO" commit -m "..."
```

Format message:

```
[global-config] <mô tả ngắn>

- <chi tiết 1>
- <chi tiết 2>

Sửa từ project: <tên project đang làm khi phát sinh thay đổi>
```

Bước 5 — Push (hỏi user trước)

```
git -C "$REPO" log --oneline origin/main..HEAD    # xem sắp push gì
git -C "$REPO" push
```

BẮT BUỘC hỏi user trước khi push. Nếu đang ở nhánh không phải `main` → nói rõ tên nhánh.

Bước 6 — Báo cáo

```
✓ Config dùng chung đã đồng bộ
Repo      : <đường dẫn>
Nhánh     : <branch> (<n> commit ahead/sạch)
Đã commit: <danh sách file>
Push      : <đã push / user từ chối / không có gì để push>
```

Quy tắc cứng

1. **KHÔNG** `git add -A` — chỉ stage file mình thực sự sửa.
2. **KHÔNG** tự push khi user chưa xác nhận.
3. **KHÔNG** tự resolve conflict — dừng và báo.
4. Nếu file `.md` vừa sửa cần xuất DOCX/PDF (§19) → nhớ rằng `.claude/**/*.*docx|pdf` đã gitignore (phương án b), nên xuất ra là file cục bộ, **KHÔNG** commit — dừng cố `git add` chúng.
5. Sửa xong agent/skill có đổi `description` frontmatter → gợi ý user chạy `/skill-trigger-test` để xác nhận Dispatcher vẫn route đúng.